Aula 07

Banco de dados com SQLite

Seus dados finalmente sobrevivem ao restart

Um teste rápido

1. Crie uma tarefa pela API ✅
2. Reinicie o servidor 🔄
3. Liste as tarefas...

😱 Sumiu.

```
let tarefas = [ ... ];   // isso vive na RAM
```

O que precisamos

Requisito	Por quê
Sobreviver ao restart	dados são o ativo do sistema
Buscar rápido em 1 milhão	`array.find()` não escala
Vários acessos ao mesmo tempo	um servidor atende muita gente
Garantir integridade	não existe pedido sem cliente

= Banco de dados

Banco relacional = planilha com regras

TABELA: tarefas

id	titulo	concluida	prioridade
1	Estudar Node.js	1	alta
2	Aprender Express	0	alta

← COLUNAS

← LINHA

Vocabulário

Termo	O que é
Tabela	coleção de coisas do mesmo tipo
Coluna	um campo, com tipo definido
Linha	um registro
Chave primária	identifica a linha (o `id`)
Chave estrangeira	aponta para outra tabela

← É daí que vem o "relacional"

Por que SQLite?

	SQLite	PostgreSQL
Instalação	**nenhuma**	servidor + config
Onde vive	um arquivo `.db`	processo separado
Bom para	aprender, apps pequenos	produção

O **SQL é quase o mesmo** nos dois.

Aprender SQLite hoje = aprender Postgres amanhã.

CREATE TABLE

```
CREATE TABLE IF NOT EXISTS tarefas (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  titulo      TEXT    NOT NULL,
  concluida   INTEGER NOT NULL DEFAULT 0,
  prioridade TEXT    NOT NULL DEFAULT 'media',
  criada_em   TEXT    NOT NULL
);
```

| `PRIMARY KEY` | identifica, não repete |

| `AUTOINCREMENT` | o banco gera o número |

| `NOT NULL` | obrigatório |

| `DEFAULT` | valor se ninguém informar |

⚠️ O SQLite não tem boolean

Use `INTEGER`:

`0` = falso · `1` = verdadeiro

Datas também são `TEXT`, em formato ISO.

A conversão acontece **no repository**,
e em mais lugar nenhum.

SELECT

```sql
SELECT * FROM tarefas;
SELECT id, titulo FROM tarefas;
SELECT * FROM tarefas WHERE concluida = 0;
SELECT * FROM tarefas WHERE titulo LIKE '%node%';
SELECT * FROM tarefas ORDER BY criada_em DESC;
SELECT * FROM tarefas LIMIT 10;
SELECT COUNT(*) AS total FROM tarefas;
```

É o `filter`, o `sort` e o `length` da aula 02 — executados pelo banco, muito mais rápido.

INSERT · UPDATE · DELETE

```sql
INSERT INTO tarefas (titulo, criada_em)
VALUES ('Estudar SQL', '2026-04-01T10:00:00Z');

UPDATE tarefas SET concluida = 1 WHERE id = 3;

DELETE FROM tarefas WHERE id = 3;
```

🚨 O erro mais caro do SQL

```
UPDATE tarefas SET concluida = 1;    -- TODAS
DELETE FROM tarefas;                 -- TUDO
```

Esqueceu o `WHERE`.

Um dia você vai fazer isso.

Que seja num banco de teste.

> **Dica:** escreva o `WHERE` primeiro, depois volte e escreva o resto.

SQLite no Node — 3 métodos

```
import { DatabaseSync } from "node:sqlite";
const db = new DatabaseSync("./dados.db");
```

Método	Quando	Devolve
`.all()`	várias linhas	array
`.get()`	uma linha	objeto ou `undefined`
`.run()`	INSERT/UPDATE/DELETE	`{ changes, lastInsertRowid }`

Por que `prepare` separado?

```
const inserir = db.prepare(
  "INSERT INTO tarefas (titulo) VALUES (?)"
);

for (const t of lista) inserir.run(t.titulo);
//    prepara 1x, executa N vezes
```

Mais rápido — e é o que permite usar `?` com segurança.

E o `db.exec()`?

```
db.exec("CREATE TABLE ...");     // sem retorno
db.exec("BEGIN TRANSACTION");
```

| `exec` | comandos sem retorno |

| `prepare` | quando há valores ou resultado |

`changes` é ouro

```
const r = db.prepare("UPDATE ... WHERE id = ?").run(id);

if (r.changes === 0) {
  // não existe linha com esse id
}
```

É assim que o repository sabe devolver `null`

e o service sabe lançar 404.

🚨 A parte séria da aula

SQL Injection

❌❌❌ NUNCA faça isso

```
const busca = req.query.busca;

db.prepare(
  `SELECT * FROM tarefas WHERE titulo = '${busca}'`
).all();
```

O que um atacante envia

```
busca = '; DROP TABLE tarefas; --
```

O comando executado vira:

```
SELECT * FROM tarefas WHERE titulo = '';
DROP TABLE tarefas; --'
```

Sua tabela some.

✅ SEMPRE com parâmetro

```
db.prepare("SELECT * FROM tarefas WHERE titulo = ?")
  .all(busca);
```

Com `?`, o banco trata o valor **como dado**, nunca como comando.

Mesmo que o texto contenha `DROP TABLE`, será apenas um título esquisito.

Regra sem exceção

Todo valor que vem de fora entra por `?`

Sem exceção.

Nem em teste. Nem "só dessa vez".

Agora a recompensa da aula 06

Vamos trocar o array por SQLite.

Quantos arquivos mudam?

Um. Só o `repository`.

```
✏  repositories/tarefa.repository.js
✅  services/           intacto
✅  controllers/        intacto
✅  routes/             intacto
✅  validacoes/         intacto
```

← Foi para isso que separamos em camadas

O repository traduz os dois mundos

```
function paraObjeto(linha) {
  return {
    id: linha.id,
    titulo: linha.titulo,
    concluida: linha.concluida === 1,  // INTEGER → boolean
    criadaEm: linha.criada_em,         // snake → camel
  };
}
```

O banco fala `snake_case`. O JS fala `camelCase`.

Traduzir é trabalho desta camada. De mais nenhuma.

Organização

```
src/banco/
├── conexao.js    abre o .db e exporta a conexão
├── schema.sql    o CREATE TABLE
└── popular.js    dados de exemplo (seed)
```

```
*.db
```

Banco é **dado**, não código.

Vai para o Git: o `schema.sql` e o seed.

Relacionamentos

```sql
CREATE TABLE tarefas (
  id         INTEGER PRIMARY KEY AUTOINCREMENT,
  titulo     TEXT NOT NULL,
  usuario_id INTEGER NOT NULL,
  FOREIGN KEY (usuario_id)
    REFERENCES usuarios(id) ON DELETE CASCADE
);
```

O banco **recusa** tarefa de usuário inexistente.

⚠️ No SQLite: `PRAGMA foreign_keys = ON`

JOIN (você vai ver mais no M2)

```sql
SELECT tarefas.titulo, usuarios.nome
FROM tarefas
JOIN usuarios ON usuarios.id = tarefas.usuario_id;
```

Buscar de duas tabelas de uma vez.

Recapitulando

1. Memória some no restart · banco sobrevive
2. `CREATE` · `INSERT` · `SELECT` · `UPDATE` · `DELETE` — **nunca esqueça o `WHERE`**
3. Todo valor de fora entra por `?`

Mão na massa 🚀

Próxima aula: o back está pronto.

Vamos construir a **tela**. ⚛️